SAVE A COPY OF THIS NOTEBOOK TO PUT ANSWERS INTO

Part 1 (Reading Assignment), 20 points

Reminder: there are 5 reading assignments, 3% each for 15% total of your final grade.

The reading assignment consists of two papers - one on alignment before fusion, and one on the Platonic Representation Hypothesis. Read both papers and then answer the following questions:

1. Explain the implications of 'align before fuse' on your tasks of interest. at which degree, and what type of fusion needs to be performed? And what level of alignment would you need to perform for your data, so that subsequent fusion or representation learning is successful?

Referring to my group project, we aim to use 3 modalities: tabular (demographics), EEG, and ECG, in order to predict the onset of a seizure. In the first paper, the prevailing motivation behind align-before-fuse is that the image-text pairs should have a level of mutual information, and that aligning the unimodal latent representatinos before multimodal encoding allows the downstream network to better use this mutual information. I think the most obvious parallel to my project is that within the EEG and ECG data, we can expect to encode some of the same information regarding the onset of the seizure (as well as the severity of the seizure, and other auxiliary targets). In this sense, it would make sense to align the representations of the ECG and EEG in some shared latent space before multimodal encoding. Since the tabular data should NOT be expected to contain the same information as the two cardiogram modalities, it does not make sense to align this modality, instead, we can simply add this in (encoded or raw) dring the fusion step. We can try multiple types of fusion, but if we follow this align-before-fuse idea, then we should use early fusion, where we first encode each modality, combine them, then encode further. In terms of the type of fusion, as mentioned in lecture, additive fusion is a good baseline.

2. How can you perform controlled experiments to validate the types of fusion and/or alignment needed for your tasks? What are some challenges you foresee in fusing and aligning your data?

As mentioned in part (1), a good baseline is additive fusion across tabular, EKG, and ECG modalities. In order to perform controlled experiments, we should maintain a train-test split that is consistent across all fusion types/encoder models/hyperparameters, which we can then use to validate these models on the withheld data. To extend this, we could use multiple splits, to perform cross-validation, in such a way that adds more robustness to our analysis. Some of the challenges in aligning the EEG and ECG modalities are the same as those outlined in the first paper. This primary concern with the dataset in the paper is that the image-text pairings are noisy: there are many cases where text does not accurately describe the image, or where other text describes images that they are not matched to. In this sense, the EEG and ECG will be very, very noisy, and so aligning them might require more sophisticated methods. Although I am unsure about the usefulness of a momentum model to generate pseudo-targets in this case (as we have high quality labels in help guide our downstream tasks), we might require ways to denoise the dataset. Other challenges in fusing will be the stages of data incorporation. As previously mentioned, the tabular data does not represent a view of the same fundamental information, as so we will probably need to add this modality at a different stage from the EEG

and ECG data, which can be aligned in time and should hold similar electrical signals around the seizure point..

3. Explain the implications of 'platonic representation hypothesis' on your tasks of interest. Do you believe alignment between modalities would automatically emerge as models trained on your data are scaled up?

The paper on platonic representation hypothesis is less materially relevant to the tasks of interest of my teams project. The premise is that there is some underlying statistical reality that is trying to being represented by the encoding mechanisms of LLMs and vision models, and as these models get better, these representations converge to that true reality. In our project, there would probably be some alignment that emerges automatically, as the ECG and EEG data is fundamentally of the same patient, aligned in time. Using the ideas of the paper, if the patient is the underlying reality Z, we are making observations of that Z and different time steps, then taking a sequence of these time steps and predicting whether the patient underwent a seizure during that sequence. Thus, these observations (while not encoding the same information) should overlap significantly (at least, this is a hypothesis of our research). If this is true, then as we scale our model, and use the entirety of the dataset, we should see some alignment.

4. What are some reasons why alignment would not emerge i.e., counter-arguments to the platonic representation hypothesis? You are encouraged to search for follow-up works to the original paper that both support and counter the original arguments.

The main reasons that alignment would not emerge are outlined in the limitations section of the paper itself. The main problem is that these different modalities do not contain the same information. In the context of my project, the EEG contains brain wave activity, whereas ECG contains pulse activity, and therefore both will have mutually exclusive information. In this way, there will be some limit to alignment capabilities. When I looked online for other more general rebuttals, a common point was that a research problem with the hypothesis is that as models use more and more data, the data found in the datasets overlaps signficantly, to the point where it makes sense that different models find similar representations (as the model will be a function of the training data)..

5. What experiments would you propose to validate the existence and emergence of alignment in your tasks?

The most straightforward experiment would be to train a model without contrastive loss between unimodal representations of EEG and ECG data, then compare through some kernel the similarity of these embeddings for matching pairs. If this is done on a withheld test set (or via cross-validation), we should be able to measure emergent alignment between these two modalities..

6. Can you also think of some downsides of strongly (or perfectly) aligning your data modalities? How can you design experiments to validate that these risks are not present in your trained models?

As mentioned in part (4), the EEG and ECG modalities contain different exclusive information regarding the brain and heart respectively. By encouraging alignment, we might encourage the unimodal encoders to forget PMI (pointwise mutual information, as per the terminology of the second paper). Thus, when we run inference on some downstream task(predicting seizure existence and intensity), we would not be harnessing all useful information. In other words, by forcing alignment, we might miss useful unimodal information. one way to validate the existence

of these risks would be to run baselines for each modality, as well as a baseline that does not encourage alignment of modalities. If we know that unimodal models (maybe with late fusion) perform a certain way, then we see multimodal models trained with intermediate representation alignment perform worse, we know that some information has been lost during the encoding stage due to alignment pressure.

Part 2 (Homework Assignment), 100 points

Reminder: there are 5 homework assignments, 7% each for 35% total of your final grade.

For this assignment, we will finally begin playing with some of the concepts discussed in the class regarding multimodal modeling.

The first part will deal with Einsum and Tensors.

Problem 1: Tensors (5 points)

(5 pts) Let's start with tensors. A tensor represents an N-th dimensional array of numbers. In machine learning, they are used to represent data as they can efficiently represent complex data to train with. We traditionally use PyTorch as the package of choice to work with tensors. Fill in the code below with the right tensor operations. Feel free to consult the documentation and the PyTorch tutorials for help.

In [1]:
```
import torch
mat_A = torch.rand(3, 2)
mat_B = torch.rand(2, 3)
```

In [2]:
```
# Common PyTorch operations

# Adding
mat_C = mat_A + mat_B.T

# Transpose
mat_A_transpose = mat_A.T

# Matrix multiplication
mat_mult  = mat_A @ mat_B

# Element-wise multiplication
mat_mult_elm = mat_A * mat_B.T

# Create a tensor of size (4, 4) of ones
ones = torch.ones((4, 4))

# Compute mean of A
mean_A = torch.mean(mat_A, dim=(0,1))
```

Problem 2: Einsum (5 points)

(10 pts) Now lets proceed with Einsum. This is a powerful, compact notation used for expressing complex tensor operations on multi-dimensional arrays using a simple string of index labels.

Here is a quick example of using einsum to multiply two matrices.

In [3]:
```
A = torch.rand(3, 2)
B = torch.rand(2, 3)

C = torch.einsum('ij,jk->ik', A, B)
print(C)
```

```
tensor([[0.2317, 0.0313, 0.0234],
        [0.1125, 0.2365, 0.0224],
        [0.7542, 0.3955, 0.0908]])
```

The labels provide a shorthand as to what operation to do. Think of the left index as what is before, and the right as to what the dimensions of the final product should look like.

Now use this to do the other possible operations:

In [4]:
```
a = torch.rand(3,1)
b = torch.rand(3,1)

A = torch.rand(3, 2)
B = torch.rand(2, 3)

# Dot Product of a and b
d_prod = torch.einsum('ij,ij', a, b)

# Transpose using vector b
transpose = torch.einsum('ji', b)

# Summation (element-wise and column-wise of A)
sum_element = torch.einsum('ij->', A)
sum_column = torch.einsum('ij->j', A)

# Diagonal of A
diag = torch.einsum('ii->i', A[:2, :2])

# Outer Product of a and b
outer = torch.einsum('i,j->ij', a.squeeze(), b.squeeze())
```

In [5]:
```
# Tests to verify that operations were done correctly
def to_list(t):
    return t.detach().cpu().tolist()

def check_dot_product(ans, a, b):
    expected = sum(i[0] * j[0] for i, j in zip(to_list(a), to_list(b)))
    assert torch.allclose(ans, torch.tensor(expected))

def check_transpose(ans, b):
    b_list = to_list(b)
    expected = [[row[i] for row in b_list] for i in range(len(b_list[0]))]
    assert torch.allclose(ans, torch.tensor(expected))

def check_sum_element(ans, A):
    expected = sum(val for row in to_list(A) for val in row)
    assert torch.allclose(ans, torch.tensor(expected))

def check_sum_column(ans, A):
    A_list = to_list(A)
    expected = [sum(row[i] for row in A_list) for i in range(len(A_list[0]))]
    assert torch.allclose(ans, torch.tensor(expected))
```

```
def check_diagonal(ans, A):
    A_list = to_list(A)
    expected = [A_list[i][i] for i in range(min(len(A_list), len(A_list[0])))]
    assert torch.allclose(ans, torch.tensor(expected))

def check_outer_product(ans, a, b):
    a_l, b_l = to_list(a.squeeze(1)), to_list(b.squeeze(1))
    expected = [[i * j for j in b_l] for i in a_l]
    assert torch.allclose(ans, torch.tensor(expected))
```

In [6]:
```
check_dot_product(d_prod, a, b)
check_transpose(transpose, b)
check_sum_element(sum_element, A)
check_sum_column(sum_column, A)
check_diagonal(diag, A)
check_outer_product(outer, a, b)

print("NO PROBLEMS!!!!")
```

```
NO PROBLEMS!!!!
```

Problem 3: Unimodal Models (10 points)

We now explore unimodal models and multimodal fusion. For the first part we will work on the image and audio digit dataset AV-MNIST to do digit classification. To benchmark effectiveness, we will use the Multibench benchmark. First, we will clone the repo, and get the necessary packages and dataset.

Note: MAKE SURE YOU SWITCH TO A GPU TO RUN THE MODELS. RUNTIME -> CHANGE RUNTIME TYPE -> T4 GPU (or any other). Be mindful of Google's GPU limits based on what kind of account you own.

Also, if you are a student you should be able to have Colab Pro for free if you don't already. Take advantage of that!

THIS IS AN EXAMPLE, DO NOT BE RESTRICTED BY WHAT WE DO HERE WHEN YOU HAVE TO IMPLEMENT THIS FOR YOUR OWN DATASET.

Getting repo

In [1]:
```
!git clone https://github.com/pliang279/MultiBench.git
%cd MultiBench
```

```
fatal: destination path 'MultiBench' already exists and is not an empty directory.
/content/MultiBench
```

Getting AV-MNIST dataset

In [2]:
```
!mkdir data
!pip install gdown
!pip install torch==2.3.1 torchvision==0.18.1 torchtext==0.18.0 torchaudio==2.3.1
!pip install memory_profiler
```

```
mkdir: cannot create directory ‘data’: File exists
Requirement already satisfied: gdown in /usr/local/lib/python3.12/dist-packages (5.2.1)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-packages
(from gdown) (4.13.5)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from
gdown) (3.24.3)
Requirement already satisfied: requests[socks] in /usr/local/lib/python3.12/dist-packages
(from gdown) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from gdow
n) (4.67.3)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages
(from beautifulsoup4->gdown) (2.8.3)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist
-packages (from beautifulsoup4->gdown) (4.15.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist
-packages (from requests[socks]->gdown) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (f
rom requests[socks]->gdown) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packa
ges (from requests[socks]->gdown) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packa
ges (from requests[socks]->gdown) (2026.2.25)
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in /usr/local/lib/python3.12/dist-p
ackages (from requests[socks]->gdown) (1.7.1)
Requirement already satisfied: torch==2.3.1 in /usr/local/lib/python3.12/dist-packages
(2.3.1)
Requirement already satisfied: torchvision==0.18.1 in /usr/local/lib/python3.12/dist-pack
ages (0.18.1)
Requirement already satisfied: torchtext==0.18.0 in /usr/local/lib/python3.12/dist-packag
es (0.18.0)
Requirement already satisfied: torchaudio==2.3.1 in /usr/local/lib/python3.12/dist-packag
es (2.3.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from
torch==2.3.1) (3.24.3)
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.12/dist
-packages (from torch==2.3.1) (4.15.0)
Requirement already satisfied: sympy in /usr/local/lib/python3.12/dist-packages (from tor
ch==2.3.1) (1.14.0)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from
torch==2.3.1) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from to
rch==2.3.1) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from to
rch==2.3.1) (2025.3.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.1.105 in /usr/local/lib/python
3.12/dist-packages (from torch==2.3.1) (12.1.105)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.1.105 in /usr/local/lib/pytho
n3.12/dist-packages (from torch==2.3.1) (12.1.105)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.1.105 in /usr/local/lib/python
3.12/dist-packages (from torch==2.3.1) (12.1.105)
Requirement already satisfied: nvidia-cudnn-cu12==8.9.2.26 in /usr/local/lib/python3.12/d
ist-packages (from torch==2.3.1) (8.9.2.26)
Requirement already satisfied: nvidia-cublas-cu12==12.1.3.1 in /usr/local/lib/python3.12/
dist-packages (from torch==2.3.1) (12.1.3.1)
Requirement already satisfied: nvidia-cufft-cu12==11.0.2.54 in /usr/local/lib/python3.12/
dist-packages (from torch==2.3.1) (11.0.2.54)
Requirement already satisfied: nvidia-curand-cu12==10.3.2.106 in /usr/local/lib/python3.1
2/dist-packages (from torch==2.3.1) (10.3.2.106)
Requirement already satisfied: nvidia-cusolver-cu12==11.4.5.107 in /usr/local/lib/python
3.12/dist-packages (from torch==2.3.1) (11.4.5.107)
Requirement already satisfied: nvidia-cusparse-cu12==12.1.0.106 in /usr/local/lib/python
3.12/dist-packages (from torch==2.3.1) (12.1.0.106)
```

```
Requirement already satisfied: nvidia-nccl-cu12==2.20.5 in /usr/local/lib/python3.12/dist
-packages (from torch==2.3.1) (2.20.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.1.105 in /usr/local/lib/python3.12/di
st-packages (from torch==2.3.1) (12.1.105)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from tor
chvision==0.18.1) (2.0.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-pa
ckages (from torchvision==0.18.1) (11.3.0)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from torc
htext==0.18.0) (4.67.3)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from
torchtext==0.18.0) (2.32.4)
Requirement already satisfied: nvidia-nvjitlink-cu12 in /usr/local/lib/python3.12/dist-pa
ckages (from nvidia-cusolver-cu12==11.4.5.107->torch==2.3.1) (12.8.93)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages
(from jinja2->torch==2.3.1) (3.0.3)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist
-packages (from requests->torchtext==0.18.0) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (f
rom requests->torchtext==0.18.0) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packa
ges (from requests->torchtext==0.18.0) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packa
ges (from requests->torchtext==0.18.0) (2026.2.25)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packa
ges (from sympy->torch==2.3.1) (1.3.0)
Requirement already satisfied: memory_profiler in /usr/local/lib/python3.12/dist-packages
(0.61.0)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from me
mory_profiler) (5.9.5)
```

In [12]:
```
# !gdown 1KvKynJJca5tDtI5Mmp6CoRh9pQywH8Xp
!tar -xvzf avmnist.tar.gz
```

```
tar (child): avmnist.tar.gz: Cannot open: No such file or directory
tar (child): Error is not recoverable: exiting now
tar: Child returned status 2
tar: Error is not recoverable: exiting now
```

In [10]:
```
!rm -rf avmnist
```

In [13]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

In [21]:
```
!cp -r '/content/drive/MyDrive/avmnist' .
```

In [30]:
```
# 1. Path to the folder you untarred
data_dir = '/content/MultiBench/avmnist'

from datasets.avmnist.get_data import get_dataloader
traindata, validdata, testdata  = get_dataloader(data_dir, batch_size=256)
```

Getting packages

In [31]:
```
import torch
import torch.nn as nn
import torch.nn.functional as F
import sys
```

```
import os
import torch.optim as optim
from tqdm import tqdm
from unimodals.common_models import GRU, MLP, Sequential, Identity
from training_structures.Supervised_Learning import train, test
```

We will now start by creating, training, and testing unimodal models for each of the AV-MNIST modalities.

Audio

In [32]:
```
class AudioModel(nn.Module):
    def __init__(self, input_dim=12544, hidden_dim=64, dropout_probability=0.2):
        super(AudioModel, self).__init__()
        self.conv = nn.Sequential(
            # Start with a stride of 2 to instantly cut data in half
            nn.Conv2d(1, 16, kernel_size=3, stride=2, padding=1), # 112 -> 56
            nn.ReLU(),
            nn.MaxPool2d(2),                                      # 56 -> 28
            nn.Conv2d(16, 32, kernel_size=3, stride=2, padding=1), # 28 -> 14
            nn.ReLU(),
            nn.Flatten() # Only 6272 features now!
        )
        self.fc = nn.Linear(6272, 10)

    def forward(self, x):
        x = x.view(-1, 1, 112, 112)
        return self.fc(self.conv(x))
```

Image

In [33]:
```
class ImageModel(nn.Module):
    def __init__(self, dropout_prob=0.2):
        super(ImageModel, self).__init__()

        # input: [batch, 1, 28, 28]
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1

        self.pool = nn.MaxPool2d(kernel_size=2, stride=2) # Reduces size by half

        # After two poolings: 28 -> 14 -> 7
        # Final flattened size: 64 channels * 7 * 7
        self.fc = nn.Sequential(
            nn.Linear(64 * 7 * 7, 256),
            nn.ReLU(),
            nn.Dropout(dropout_prob),
            nn.Linear(256, 10)
        )

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))

        # Flatten all dimensions except batch
        x = torch.flatten(x, 1)
        return self.fc(x)
```

Training and Testing

We use cross-entropy due to this being a classification task

In [34]:
```
import torch.nn as nn
import torch.optim as optim
from torch.amp import autocast, GradScaler

# We use a scalar here to reduce system RAM use (to avoid crashing the session) while no
scaler = GradScaler()

def train_and_test_unimodal(model, train_loader, valid_loader, test_loader, modality_idx
    device = torch.device("cuda")
    model.to(device)

    # Use CrossEntropyLoss for a classification task
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=1e-3)

    best_valid_loss = float('inf')

    for epoch in range(epochs):
        model.train()
        train_loss = 0
        for batch in train_loader:
            # batch[0] = images, batch[1] = audio
            x = batch[modality_idx].to(device).float()

            # Classification labels must be Long tensors, not Float
            y = batch[2].to(device).long().squeeze()

            optimizer.zero_grad()

            with autocast(device_type='cuda'):
                outputs = model(x)
                loss = criterion(outputs, y)

            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()
            train_loss += loss.item()

        # --- Validation Phase ---
        model.eval()
        valid_loss = 0
        correct = 0
        total = 0
        with torch.no_grad():
            for batch in valid_loader:
                x = batch[modality_idx].to(device).float()
                y = batch[2].to(device).long().squeeze()

                outputs = model(x)
                valid_loss += criterion(outputs, y).item()

                _, predicted = torch.max(outputs.data, 1)
                total += y.size(0)
                correct += (predicted == y).sum().item()
```

```
        avg_train = train_loss / len(train_loader)
        avg_valid = valid_loss / len(valid_loader)
        accuracy = 100 * correct / total

        if avg_valid < best_valid_loss:
            best_valid_loss = avg_valid
            torch.save(model.state_dict(), 'best_avmnist_model.pt')

        print(f"Epoch {epoch}: Train Loss: {avg_train:.4f} | Valid Acc: {accuracy:.2f}%"

    # Final Testing follows the same logic (CrossEntropy + Index 2)
    print("\n--- Final Evaluation Complete ---")
    model.load_state_dict(torch.load('best_avmnist_model.pt'))
    model.eval()
    test_loss = 0
    correct = 0
    total = 0
    with torch.no_grad():
        for batch in test_loader:
            x = batch[modality_idx].to(device).float()
            y = batch[2].to(device).long().squeeze()

            outputs = model(x)
            test_loss += criterion(outputs, y).item()

            _, predicted = torch.max(outputs.data, 1)
            total += y.size(0)
            correct += (predicted == y).sum().item()

        test_accuracy = 100 * correct / total
        test_loss /= len(test_loader)
        print(f"Final Test Loss: {test_loss:.4f} | Test Accuracy: {test_accuracy:.2f}%")
```

Training and testing for each modality:

Audio:

In [74]:
```
s = time.time()
audio_model = AudioModel(hidden_dim=128, dropout_probability=0.25)
train_and_test_unimodal(audio_model, traindata, validdata, testdata, epochs=10, modality
print(time.time() - s)
```

```
Epoch 0: Train Loss: 2.1092 | Valid Acc: 38.16%
Epoch 1: Train Loss: 2.0169 | Valid Acc: 39.42%
Epoch 2: Train Loss: 1.9936 | Valid Acc: 40.60%
Epoch 3: Train Loss: 1.9787 | Valid Acc: 40.98%
Epoch 4: Train Loss: 1.9698 | Valid Acc: 41.20%
Epoch 5: Train Loss: 1.9610 | Valid Acc: 41.34%
Epoch 6: Train Loss: 1.9534 | Valid Acc: 41.80%
Epoch 7: Train Loss: 1.9483 | Valid Acc: 41.68%
Epoch 8: Train Loss: 1.9437 | Valid Acc: 41.64%
Epoch 9: Train Loss: 1.9381 | Valid Acc: 41.98%

--- Final Evaluation Complete ---
Final Test Loss: 2.0203 | Test Accuracy: 40.49%
52.635075092315674
```

In [75]:
```
sum(p.numel() for p in audio_model.parameters())
```

Out[75]: 67530

Image:

In [76]:
```
s = time.time()
image_model = ImageModel(dropout_prob=0.25)
train_and_test_unimodal(image_model, traindata, validdata, testdata, epochs=10, modality
print(time.time() - s)
```

```
Epoch 0: Train Loss: 1.1118 | Valid Acc: 66.18%
Epoch 1: Train Loss: 0.9356 | Valid Acc: 68.86%
Epoch 2: Train Loss: 0.9176 | Valid Acc: 67.46%
Epoch 3: Train Loss: 0.9093 | Valid Acc: 68.22%
Epoch 4: Train Loss: 0.9026 | Valid Acc: 68.82%
Epoch 5: Train Loss: 0.8996 | Valid Acc: 68.72%
Epoch 6: Train Loss: 0.8950 | Valid Acc: 68.98%
Epoch 7: Train Loss: 0.8939 | Valid Acc: 68.60%
Epoch 8: Train Loss: 0.8939 | Valid Acc: 68.10%
Epoch 9: Train Loss: 0.8893 | Valid Acc: 68.68%

--- Final Evaluation Complete ---
Final Test Loss: 0.9060 | Test Accuracy: 64.37%
32.58402180671692
```

In [77]:
```
sum(p.numel() for p in image_model.parameters())
```

Out[77]: 824458

Answer the following questions:

1. (5 points) Try to get the best performance out of each model by playing around with hyperparameters (hint: you may have to playing around and even add additional arguments to the layers like dropout, look at the documentation and look into how we can improve performace). List the best performance you were able to get and the hyperparameters you used.

   > **Audio**: dropout=0.25, hidden dim=128, epochs=10
   >
   > Final Test Loss: 2.0116 | Test Accuracy: 41.23%

   > **Visual**: 0.25 dropout, hidden dim=256, epochs=10
   >
   > Final Test Loss: 0.8941 | Test Accuracy: 64.18%

2. (5 points) Compare the performances of each modality. What do these suggest to you? What could be done to get the worst performing ones to get closer to the best performing modality/model?

The image modality significantly outperforms the audio modality. This suggests that the audio modality has less easily extracted information when compared to image. We could try to expand the audio model, and train longer, to recover some performance made by scale. In addition, we could attempt to align intermediate representations of the audio model to the image, since they should capture different "views" (observations) of the same event (the image being a number).

Problem 4: Multimodal Fusion (10 points)

Now you will play with multimodal fusion. Lets use a late fusion to improve our performance. We have provided some code with the hyperparameters to consider, but you are encouraged to play with them to try to get improvments. To make things simpler, the encoders for both modalities have been provided. However, some other parts are missing, so you will have to fill those in!

In [38]:
```
import torch.nn as nn
from unimodals.common_models import MLP
from fusions.common_fusions import MultiplicativeInteractions2Modal, Concat
from training_structures.Supervised_Learning import train

# Image Encoder
class ImageEncoder(nn.Module):
    def __init__(self, output_dim=64):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Flatten(),
            nn.Linear(64 * 7 * 7, output_dim)
        )
    def forward(self, x):
        return self.conv(x)

# Audio Encoder
class AudioEncoder(nn.Module):
    def __init__(self, output_dim=64):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 32, 3, stride=2, padding=1), nn.BatchNorm2d(32), nn.ReLU(), # 1
            nn.MaxPool2d(2), # 56 -> 28
            nn.Conv2d(32, 64, 3, stride=2, padding=1), nn.BatchNorm2d(64), nn.ReLU(), #
            nn.Flatten(),
            nn.Linear(64 * 14 * 14, output_dim)
        )
    def forward(self, x):
        return self.conv(x)

encoders = [ImageEncoder(output_dim=64).cuda(), AudioEncoder(output_dim=64).cuda()]
fusion = Concat()
head = MLP(indim=128, hiddim=256, outdim=10, dropout=True, dropoutp=0.2)

# Run Training
print("Starting Training...")
train(encoders, fusion, head, traindata, validdata, 5,
      task="classification", optimtype=torch.optim.AdamW, is_packed=False,
      lr=5e-4, save='avmnist_lmf.pt', weight_decay=0.005,
      objective=torch.nn.CrossEntropyLoss())

# Run Test
model = torch.load('avmnist_lmf.pt').cuda()
test(model, testdata, 'avmnist', is_packed=False, task="classification",
      criterion=torch.nn.CrossEntropyLoss(), no_robust=True)
```

```
Starting Training...
Epoch 0 train loss: tensor(2.9697, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 0 valid loss: tensor(1.7366, device='cuda:0') acc: 0.6654
Saving Best
Epoch 1 train loss: tensor(2.7526, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 1 valid loss: tensor(1.6462, device='cuda:0') acc: 0.7194
Saving Best
Epoch 2 train loss: tensor(2.7245, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 2 valid loss: tensor(1.6784, device='cuda:0') acc: 0.7236
Saving Best
Epoch 3 train loss: tensor(2.6985, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 3 valid loss: tensor(1.6913, device='cuda:0') acc: 0.7082
Epoch 4 train loss: tensor(2.6783, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 4 valid loss: tensor(1.6905, device='cuda:0') acc: 0.719
Training Time: 39.88918590545654
Training Peak Mem: 9869.98046875
Training Params: 1077258
acc: 0.6892
Inference Time: 1.208221673965454
Inference Params: 1077258
```

Answer the following:

1. (2 points) Sometimes when training you may notice the model gets stuck in a range of loss and never seems to get it's loss down. What does this suggest? What are some ways you can fix this?

This could be from several different factors, including bad initialization, low learning rate, or vanishing gradients. To mitigate these risks, we could add dropout, run multiple times (for random initialization), and add normalizations. I did not have the model getting stuck when I ran this, so it is hard to comment further.

2. (2 points) What are some other fusion methods we could use that we could use? Would they lead to improvements compared to early fusion?

We could use any of the fusion methods covered in class, including additive, multiplicative, tensor, or low-rank tensor. By concatenating, we preserve the information, but the intermediate dimensionality is high. This is also true for multiplicative or tensor fusion methods, but in these, we can harness more effective cross-modality effects. These might lead to improvements because of this.

3. (6 points) Explain the difference between early fusion techniques and late fusion techniques. Be sure to discuss their benefits and tradeoffs.

Early fusion combines intermediate embeddings of each modality, which is than passed to another MLP or prediction head. Late fusion instead predicts using unimodal modals, then combines the predictions (logits, etc) in fusion. Early fusion allows modalities to attend to eachother, which can improve cross-modality effects, whereas late fusion might be good for simpler problems, or in cases of ensembling multiple types of models that you want to keep separate.

Problem 5: Other Fusion Techniques (30 points)

Now, we want you to try implementing some of these fusion techniques on your dataset! For this part, you will implement these fusion techniques:

1. Early fusion
2. Late fusion
3. TensorFusion
4. Low-Rank Tensor (LMF) Fusion

You cannot just import and use the functions available in Multibench to do this. In addition, use einsum where applicable. TO RECIEVE FULL CREDIT, THE FUSIONS YOU IMPLEMENTATION MUST WORK WITH THE DATASET YOU CREATED FROM HOMEWORK 1. YOU WILL HAVE TO CREATE A SIMPLE MODEL FOR EACH FUSION TECHNIQUE TO PLAY WITH.

(5 points) In your write up, report the best validation accuracies of your multimodal model (don't forget to include what hyperparameters you included) after training and any modifications that had to be done to your data or model to be able to train on it. In addition, talk about which technique you believe would be best for your dataset and why that is.

Design the fusion classes with the modalities you are specifically working with in mind. The example we worked through above with MOSI was meant as a showcase of fusion in action - we do not require you to use text, video and audio as the modalities. Use whichever ones you are working with!

The code below provided is to be filled in with the models you set up for each technique. For an example, the first fusion technique has been done for you.

Answer Here:

NOTE FOR GRADER: in homework 1, I chose a report+image dataset ONLY, with no labels. Therefore, it is hard to use fusion. To get around this, I am just using the multimodal dataset of part (3) and (4).

In [65]:
```
# Imports incase you need them again! Feel free to include anything else you need
import torch
from torch import nn
from torch.nn import functional as F
import pdb
from torch.autograd import Variable
import time

from training_structures.Supervised_Learning import train
from unimodals.common_models import MLP, Identity
```

In [41]:
```
import pandas as pd

results = pd.DataFrame(columns=[
    "fusion_type",
    "num_params",
    "memory_mb",
    "time_to_converge_sec",
])
```

Early Fusion

In [53]:
```
class EarlyFusion(nn.Module):
  def __init__(self):
    super(EarlyFusion, self).__init__()

  def forward(self, x):
    return torch.einsum('bi,bj->bij', x[0], x[1]).flatten(1,-1)
```

In [58]:
```
encoders = [ImageEncoder(output_dim=64).cuda(), AudioEncoder(output_dim=64).cuda()]
fusion = EarlyFusion()
head = MLP(indim=4096, hiddim=256, outdim=10, dropout=True, dropoutp=0.2)


# Run Training
print("Starting Training...")
start_time = time.time()
train(encoders, fusion, head, traindata, validdata, 5,
      task="classification", optimtype=torch.optim.AdamW, is_packed=False,
      lr=5e-4, save='avmnist_early.pt', weight_decay=0.005,
      objective=torch.nn.CrossEntropyLoss())
end_time = time.time()

# Run Test
model = torch.load('avmnist_early.pt').cuda()
test(model, testdata, 'avmnist', is_packed=False, task="classification",
      criterion=torch.nn.CrossEntropyLoss(), no_robust=True)

p = sum(p.numel() for p in model.parameters())
new_row = {'model': 'early', 'num_params': p, 'time_to_converge_sec': end_time-start_tim
results.loc[len(results)] = new_row
```

```
Starting Training...
Epoch 0 train loss: tensor(3.1021, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 0 valid loss: tensor(1.3987, device='cuda:0') acc: 0.689
Saving Best
Epoch 1 train loss: tensor(2.7918, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 1 valid loss: tensor(1.4286, device='cuda:0') acc: 0.7174
Saving Best
Epoch 2 train loss: tensor(2.7587, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 2 valid loss: tensor(1.5937, device='cuda:0') acc: 0.7114
Epoch 3 train loss: tensor(2.7235, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 3 valid loss: tensor(1.7082, device='cuda:0') acc: 0.7134
Epoch 4 train loss: tensor(2.6908, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 4 valid loss: tensor(1.4917, device='cuda:0') acc: 0.7244
Saving Best
Training Time: 41.21642708778381
Training Peak Mem: 10034.078125
Training Params: 2093066
acc: 0.6868
Inference Time: 1.211428165435791
Inference Params: 2093066
```

(5 Points) Late Fusion

In [66]:
```
class LateFusion(nn.Module):
  def __init__(self):
    super(LateFusion, self).__init__()
    self.fc0 = nn.Linear(64, 10)
    self.fc1 = nn.Linear(64, 10)
    self.w0 = torch.nn.Parameter(torch.ones(10))
```

```
    self.w1 = torch.nn.Parameter(torch.ones(10))

  def forward(self, x):
    return torch.einsum('bc,c->bc', self.fc0(x[0]), self.w0) + torch.einsum('bc,c->bc',
```

In [67]:
```
encoders = [ImageEncoder(output_dim=64).cuda(), AudioEncoder(output_dim=64).cuda()]
fusion = LateFusion()
head = Identity()

# Run Training
print("Starting Training...")
start_time = time.time()
train(encoders, fusion, head, traindata, validdata, 5,
      task="classification", optimtype=torch.optim.AdamW, is_packed=False,
      lr=5e-4, save='avmnist_late.pt', weight_decay=0.005,
      objective=torch.nn.CrossEntropyLoss())
end_time = time.time()

# Run Test
model = torch.load('avmnist_late.pt').cuda()
test(model, testdata, 'avmnist', is_packed=False, task="classification",
      criterion=torch.nn.CrossEntropyLoss(), no_robust=True)

p = sum(p.numel() for p in model.parameters())
new_row = {'model': 'late', 'num_params': p, 'time_to_converge_sec': end_time-start_time
results.loc[len(results)] = new_row
```

```
Starting Training...
Epoch 0 train loss: tensor(0.9714, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 0 valid loss: tensor(0.7694, device='cuda:0') acc: 0.7084
Saving Best
Epoch 1 train loss: tensor(0.7882, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 1 valid loss: tensor(0.7483, device='cuda:0') acc: 0.7212
Saving Best
Epoch 2 train loss: tensor(0.7410, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 2 valid loss: tensor(0.7364, device='cuda:0') acc: 0.7292
Saving Best
Epoch 3 train loss: tensor(0.7110, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 3 valid loss: tensor(0.7513, device='cuda:0') acc: 0.7254
Epoch 4 train loss: tensor(0.7023, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 4 valid loss: tensor(0.7450, device='cuda:0') acc: 0.7232
Training Time: 39.71893572807312
Training Peak Mem: 9032.8984375
Training Params: 1042984
acc: 0.704
Inference Time: 1.2944512367248535
Inference Params: 1042984
```

(5 points) Tensor Fusion

In [69]:
```
class TensorFusion(nn.Module):
  def __init__(self):
    super(TensorFusion, self).__init__()
    self.fc0 = nn.Linear(64, 10)
    self.fc1 = nn.Linear(64, 10)
    self.fc_final = nn.Linear(121, 10)

  def forward(self, x):
    B, D = x[0].shape
    ones = torch.ones(B, 1, device=x[0].device)
```

```
    x0_aug = torch.cat([ones, self.fc0(x[0])], dim=1)   # (B, D+1)
    x1_aug = torch.cat([ones, self.fc1(x[1])], dim=1)   # (B, D+1)

    out = torch.einsum('bi,bj->bij', x0_aug, x1_aug)
    out = out.reshape(B, -1)
    return self.fc_final(out) # flatten out for next layer
```

In [70]:
```
encoders = [ImageEncoder(output_dim=64).cuda(), AudioEncoder(output_dim=64).cuda()]
fusion = TensorFusion()
head = Identity()

# Run Training
print("Starting Training...")
start_time = time.time()
train(encoders, fusion, head, traindata, validdata, 5,
      task="classification", optimtype=torch.optim.AdamW, is_packed=False,
      lr=5e-4, save='avmnist_tensor.pt', weight_decay=0.005,
      objective=torch.nn.CrossEntropyLoss())
end_time = time.time()

# Run Test
model = torch.load('avmnist_tensor.pt').cuda()
test(model, testdata, 'avmnist', is_packed=False, task="classification",
      criterion=torch.nn.CrossEntropyLoss(), no_robust=True)

p = sum(p.numel() for p in model.parameters())
new_row = {'model': 'tensor', 'num_params': p, 'time_to_converge_sec': end_time-start_ti
results.loc[len(results)] = new_row
```

```
Starting Training...
Epoch 0 train loss: tensor(0.9975, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 0 valid loss: tensor(0.7938, device='cuda:0') acc: 0.7014
Saving Best
Epoch 1 train loss: tensor(0.8163, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 1 valid loss: tensor(0.7714, device='cuda:0') acc: 0.7154
Saving Best
Epoch 2 train loss: tensor(0.7614, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 2 valid loss: tensor(0.7541, device='cuda:0') acc: 0.7196
Saving Best
Epoch 3 train loss: tensor(0.7396, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 3 valid loss: tensor(0.7756, device='cuda:0') acc: 0.715
Epoch 4 train loss: tensor(0.7124, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 4 valid loss: tensor(0.7618, device='cuda:0') acc: 0.7212
Saving Best
Training Time: 39.26295852661133
Training Peak Mem: 9006.99609375
Training Params: 1044184
acc: 0.6988
Inference Time: 1.2051448822021484
Inference Params: 1044184
```

(5 Points) Low-Rank Tensor Fusion (LMF) Fusion

In [71]:
```
class LMFFusion(nn.Module):
  def __init__(self, embed_dim, num_layers):
    super(LMFFusion, self).__init__()

  def forward(self, x):
    pass
```

(10 points) In addition, create some visualizations of the following for each fusion:

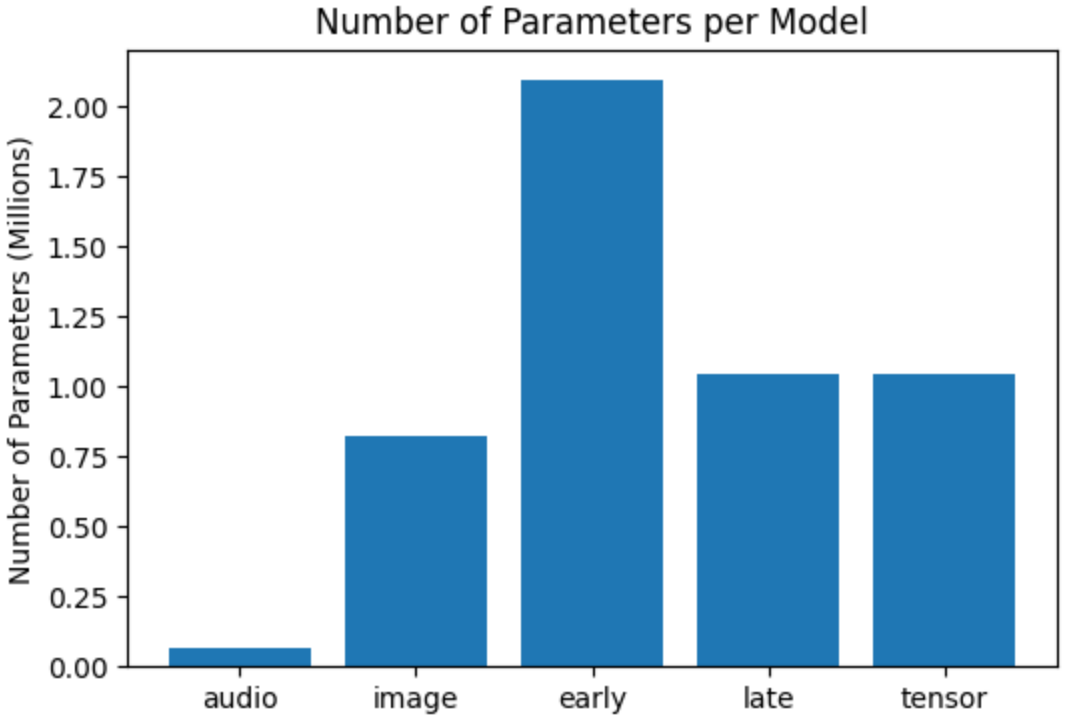
- Number of parameters for each model (unimodal and multimodal)
- Memory Use
- Time until convergence

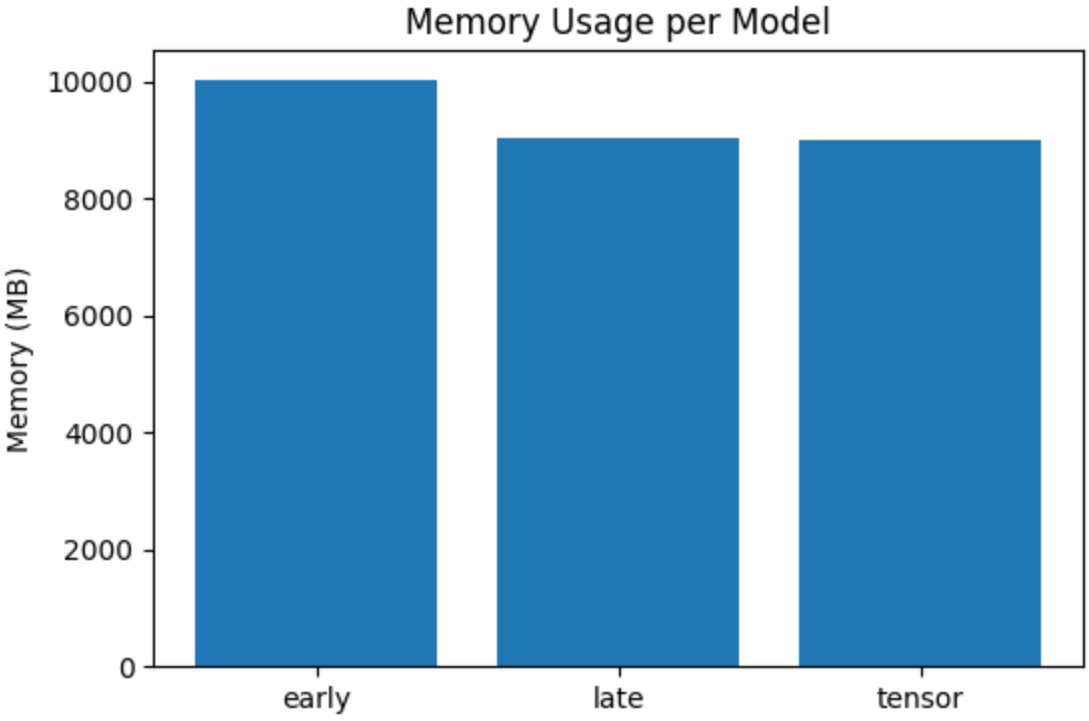
You are free to plot them here or through other means (like wandb). After doing so, discuss what are the pros and cons of unimodal versus multimodal models.

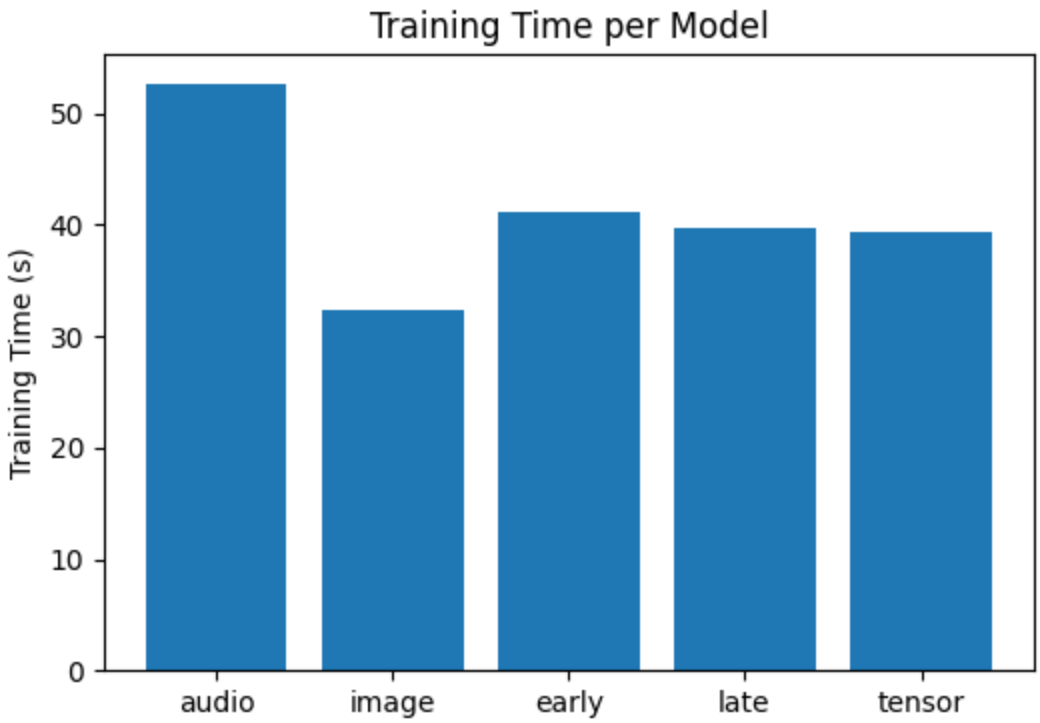
I was not able to configure the unimodal models to get memory usage during training. In general, as expected, unimodal models are smaller, such as the audio, and (can be) faster to train, such as the image model. The multimodal models have much more parameters, especially when doing large operations like the early fusion method, which creates a 64x64 latent representation, which then requires a large linear layer to convert back to a workable hidden dim. In terms of memory, the late and tensor methods are much better, and are also slightly faster to train, probably due to the smaller number of parameters compared to early fusion.

In [78]:
```
results = pd.DataFrame([
    {'model': 'audio', 'time_sec': 52.635, 'memory_mb': None, 'num_params': 67530},
    {'model': 'image', 'time_sec': 32.4, 'memory_mb': None, 'num_params': 824458},
    {'model': 'early', 'time_sec': 41.21, 'memory_mb': 10034.07, 'num_params': 2093066},
    {'model': 'late', 'time_sec': 39.71, 'memory_mb': 9032.89, 'num_params': 1042984},
    {'model': 'tensor', 'time_sec': 39.262, 'memory_mb': 9006.9, 'num_params': 1044184}
])
```

In [79]:
```
import matplotlib.pyplot as plt

# Plot Number of Parameters
plt.figure(figsize=(6,4))
plt.bar(results['model'], results['num_params']/1e6)
plt.ylabel("Number of Parameters (Millions)")
plt.title("Number of Parameters per Model")
plt.show()

# Plot Memory Use
plt.figure(figsize=(6,4))
plt.bar(results['model'], results['memory_mb'])
plt.ylabel("Memory (MB)")
plt.title("Memory Usage per Model")
plt.show()

# Plot Training Time
plt.figure(figsize=(6,4))
plt.bar(results['model'], results['time_sec'])
plt.ylabel("Training Time (s)")
plt.title("Training Time per Model")
plt.show()
```


Number of Parameters per Model
Number of Parameters (Millions)
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
audio
image
early
late
tensor


Memory Usage per Model
Memory (MB)
10000
8000
6000
4000
2000
0
early
late
tensor



Problem 6: Contrastive Learning (30 points)

For the next part of this HW, we will focus on contrastive learning. As a reminder, contrastive learning is a local, discrete alignment method used in machine learning. To explore this, we look at CLIP, a multimodal model developed by OpenAI that uses contrastive learning to align visual and textual data together.

THIS IS JUST AN EXAMPLE, DO NOT LET THIS RESTRICT THE IMPLEMENTATION YOU WILL BE DOING.

In [80]:
```
!pip install ftfy regex tqdm
!pip install git+https://github.com/openai/CLIP.git
```

```
Collecting ftfy
  Downloading ftfy-6.3.1-py3-none-any.whl.metadata (7.3 kB)
Requirement already satisfied: regex in /usr/local/lib/python3.12/dist-packages (2025.11.
3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (4.67.3)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.12/dist-packages (from f
tfy) (0.6.0)
Downloading ftfy-6.3.1-py3-none-any.whl (44 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.8/44.8 kB 2.1 MB/s eta 0:00:00
Installing collected packages: ftfy
Successfully installed ftfy-6.3.1
Collecting git+https://github.com/openai/CLIP.git
  Cloning https://github.com/openai/CLIP.git to /tmp/pip-req-build-lcdh1_d0
  Running command git clone --filter=blob:none --quiet https://github.com/openai/CLIP.git
/tmp/pip-req-build-lcdh1_d0
  Resolved https://github.com/openai/CLIP.git to commit ded190a052fdf4585bd685cee5bc96e03
10d2c93
  Preparing metadata (setup.py) ... done
Requirement already satisfied: ftfy in /usr/local/lib/python3.12/dist-packages (from clip
==1.0) (6.3.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from
clip==1.0) (26.0)
Requirement already satisfied: regex in /usr/local/lib/python3.12/dist-packages (from cli
p==1.0) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from clip
==1.0) (4.67.3)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (from cli
p==1.0) (2.3.1)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (fr
om clip==1.0) (0.18.1)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.12/dist-packages (from f
tfy->clip==1.0) (0.6.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from
torch->clip==1.0) (3.24.3)
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.12/dist
-packages (from torch->clip==1.0) (4.15.0)
Requirement already satisfied: sympy in /usr/local/lib/python3.12/dist-packages (from tor
ch->clip==1.0) (1.14.0)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from
torch->clip==1.0) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from to
rch->clip==1.0) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from to
rch->clip==1.0) (2025.3.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.1.105 in /usr/local/lib/python
3.12/dist-packages (from torch->clip==1.0) (12.1.105)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.1.105 in /usr/local/lib/pytho
n3.12/dist-packages (from torch->clip==1.0) (12.1.105)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.1.105 in /usr/local/lib/python
3.12/dist-packages (from torch->clip==1.0) (12.1.105)
Requirement already satisfied: nvidia-cudnn-cu12==8.9.2.26 in /usr/local/lib/python3.12/d
ist-packages (from torch->clip==1.0) (8.9.2.26)
Requirement already satisfied: nvidia-cublas-cu12==12.1.3.1 in /usr/local/lib/python3.12/
dist-packages (from torch->clip==1.0) (12.1.3.1)
Requirement already satisfied: nvidia-cufft-cu12==11.0.2.54 in /usr/local/lib/python3.12/
dist-packages (from torch->clip==1.0) (11.0.2.54)
Requirement already satisfied: nvidia-curand-cu12==10.3.2.106 in /usr/local/lib/python3.1
2/dist-packages (from torch->clip==1.0) (10.3.2.106)
Requirement already satisfied: nvidia-cusolver-cu12==11.4.5.107 in /usr/local/lib/python
3.12/dist-packages (from torch->clip==1.0) (11.4.5.107)
Requirement already satisfied: nvidia-cusparse-cu12==12.1.0.106 in /usr/local/lib/python
3.12/dist-packages (from torch->clip==1.0) (12.1.0.106)
```

```
Requirement already satisfied: nvidia-nccl-cu12==2.20.5 in /usr/local/lib/python3.12/dist
-packages (from torch->clip==1.0) (2.20.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.1.105 in /usr/local/lib/python3.12/di
st-packages (from torch->clip==1.0) (12.1.105)
Requirement already satisfied: nvidia-nvjitlink-cu12 in /usr/local/lib/python3.12/dist-pa
ckages (from nvidia-cusolver-cu12==11.4.5.107->torch->clip==1.0) (12.8.93)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from tor
chvision->clip==1.0) (2.0.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-pa
ckages (from torchvision->clip==1.0) (11.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages
(from jinja2->torch->clip==1.0) (3.0.3)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packa
ges (from sympy->torch->clip==1.0) (1.3.0)
Building wheels for collected packages: clip
  Building wheel for clip (setup.py) ... done
  Created wheel for clip: filename=clip-1.0-py3-none-any.whl size=1369490 sha256=0b99d519
667d36843ccc3cde2865d204800708e7820881699088b5dd21ceabd7
  Stored in directory: /tmp/pip-ephem-wheel-cache-sbagpmd5/wheels/35/3e/df/3d24cbfb3b6a06
f17a2bfd7d1138900d4365d9028aa8f6e92f
Successfully built clip
Installing collected packages: clip
Successfully installed clip-1.0
```

In [81]:
```python
# Packages to import
import transformers
import torch
import clip
from PIL import Image
import requests
from io import BytesIO
```

First, we create the model.

In [82]:
```python
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Loading CLIP on {device}...")
model, preprocess = clip.load("ViT-B/32", device=device)
```

```
Loading CLIP on cuda...
100%|███████████████████████████████████████| 338M/338M [00:02<00:00, 171MiB/s]
```

Next, we will load an image to use. Note that we cannot use the MOSI dataset - we need to use raw data and the data points from the dataset already have extracted features. Upload a picture of someone smiling to use for this example (you can just find one online, save it and add to here).

In [84]:
```python
image_filename = "../smiling.png" # REPLACE WITH YOUR FILE
image = Image.open(image_filename).convert("RGB")
```

Now, we will prepare the prompt to use.

In [85]:
```python
# Options to pick from
text_options = ["a photo of a sad person", "a photo of a happy person", "a photo of an a
image_input = preprocess(image).unsqueeze(0).to(device)
text_inputs = clip.tokenize(text_options).to(device)
```

Now, let's run the inference and get the results!

In [86]:
```python
with torch.no_grad():
    image_features = model.encode_image(image_input)
```

```
    text_features = model.encode_text(text_inputs)

    # Normalize features
    image_features /= image_features.norm(dim=-1, keepdim=True)
    text_features /= text_features.norm(dim=-1, keepdim=True)

    # Calculate similarity (Dot Product)
    similarity = (100.0 * image_features @ text_features.T).softmax(dim=-1)
    values, indices = similarity[0].topk(3)

print(f"\nImage classified against: {text_options}")
print("-" * 30)
for value, index in zip(values, indices):
    print(f"{text_options[index]:>30s}: {100 * value.item():.2f}%")
```

```
Image classified against: ['a photo of a sad person', 'a photo of a happy person', 'a pho
to of an angry person']
------------------------------
     a photo of a happy person: 84.62%
    a photo of an angry person: 10.60%
       a photo of a sad person: 4.78%
```

(10 pts) We will now create, train and run zero-shot classification using contrastive learning for your own dataset. Fill in the missing information below for a generalize contrastive learning model. The training and zero-shot classification functions have been provided to you, through you may need to make slight modifications based on your dataset setup. **Design the model keeping in mind the modalities that you are specifically using. THE CLIP EXAMPLE ABOVE IS JUST TO SHOW CONTRASTIVE LEARNING IN ACTION - WE ARE NOT REQUIRING THAT YOU USE TEXT AND IMAGE AS THE MODALITIES OF CHOICE.** Try various queries, projectors, and settings on your dataset!

You must use einsum where applicable.

In [218…

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np

# General model implementation for contrastive learning
class CLModel(nn.Module):
  def __init__(self, temp):
    super(CLModel, self).__init__()
    # TODO:
    # 1. Create Encoders for modalities
    self.image_encoder = ImageEncoder(output_dim=64)
    self.audio_encoder = AudioEncoder(output_dim=64)

    # 2. Create a projector, which maps specific modality dimensions to a shared space.
    #     do this for each modality. (hint: fusions!)
    self.img_projector = nn.Linear(64, 32)
    self.aud_projector = nn.Linear(64, 32)

    # 3. Create learnable temperature (this has already been done for you)
    self.scale = nn.Parameter(torch.ones([]) * np.log(1/temp))

  def forward(self, x1, x2):
    # TODO:
    # 1. Extract the raw features
    # 2. Project them to the embedding space
    # 3. Normalize vectors and return
```

```
    # YOUR CODE HERE
    img_emb = self.image_encoder(x1)
    aud_emb = self.audio_encoder(x2)

    img_proj = self.img_projector(img_emb)
    aud_proj = self.aud_projector(aud_emb)

    img_final = F.normalize(img_proj, dim=1)
    aud_final = F.normalize(aud_proj, dim=1)
    return img_final, aud_final


# Contrastive loss. This pulls positives together and pulls negatives apart
class ContrastiveLoss(nn.Module):
  def __init__(self, model):
    super(ContrastiveLoss, self).__init__()
    # TODO: Initialize model and loss function as cross entropy loss
    self.model = model
    self.loss_fn = nn.CrossEntropyLoss()

  def forward(self, x1_emb, x2_emb):
    # TODO:
    # 1. Get the batch size (hint: you can get this
    #    from the dimensions of your embedded space)
    B = x1_emb.shape[0]

    # 2. Create similarity matrix using einsum
    sim_matrix = torch.einsum('bd,nd->bn', x1_emb, x2_emb)

    # 3. Create labels (hint: the coorect match for index i is label i)
    labels = torch.arange(B, device=x1_emb.device)

    # 4. Compute Symmetric loss (loss amongst rows + loss amongst columns)/2
    loss_i2t = self.loss_fn(sim_matrix * self.model.scale.exp(), labels)   # images -> t
    loss_t2i = self.loss_fn(sim_matrix.T * self.model.scale.exp(), labels) # text -> ima

    loss = (loss_i2t + loss_t2i) / 2
    return loss
```

In [219…

```
import torch.optim as optim
# Training function
def train_model(model, contrastive_loss, dataloader, num_epochs=5, learning_rate=3e-4, d

    optimizer = optim.Adam(model.parameters(), lr=learning_rate)

    model.to(device)
    model.train()
    print(f"Starting training for {num_epochs} epochs...")

    for epoch in range(num_epochs):
        epoch_loss = 0.0

        for batch_idx, (data_a, data_b, _) in enumerate(dataloader):
            data_a, data_b = data_a.float().to(device), data_b.float().to(device)

            optimizer.zero_grad()

            emb_a, emb_b = model(data_a, data_b)

            loss = contrastive_loss(emb_a, emb_b)
```

```
            loss.backward()

            optimizer.step()

            epoch_loss += loss.item()

        avg_loss = epoch_loss / len(dataloader)
        print(f"Epoch [{epoch+1}/{num_epochs}] | Loss: {avg_loss:.4f}")
```

In [221…

```
model = CLModel(temp=0.1)
cl_loss = ContrastiveLoss(model)

train_model(model, cl_loss, traindata, num_epochs=50, device='cuda')
```

```
Starting training for 50 epochs...
Epoch [1/50] | Loss: 5.5003
Epoch [2/50] | Loss: 5.4012
Epoch [3/50] | Loss: 5.3616
Epoch [4/50] | Loss: 5.3359
Epoch [5/50] | Loss: 5.3156
Epoch [6/50] | Loss: 5.2962
Epoch [7/50] | Loss: 5.2784
Epoch [8/50] | Loss: 5.2633
Epoch [9/50] | Loss: 5.2499
Epoch [10/50] | Loss: 5.2371
Epoch [11/50] | Loss: 5.2244
Epoch [12/50] | Loss: 5.2133
Epoch [13/50] | Loss: 5.2016
Epoch [14/50] | Loss: 5.1934
Epoch [15/50] | Loss: 5.1830
Epoch [16/50] | Loss: 5.1730
Epoch [17/50] | Loss: 5.1650
Epoch [18/50] | Loss: 5.1556
Epoch [19/50] | Loss: 5.1470
Epoch [20/50] | Loss: 5.1375
Epoch [21/50] | Loss: 5.1295
Epoch [22/50] | Loss: 5.1205
Epoch [23/50] | Loss: 5.1128
Epoch [24/50] | Loss: 5.1052
Epoch [25/50] | Loss: 5.0956
Epoch [26/50] | Loss: 5.0883
Epoch [27/50] | Loss: 5.0801
Epoch [28/50] | Loss: 5.0722
Epoch [29/50] | Loss: 5.0667
Epoch [30/50] | Loss: 5.0600
Epoch [31/50] | Loss: 5.0537
Epoch [32/50] | Loss: 5.0462
Epoch [33/50] | Loss: 5.0396
Epoch [34/50] | Loss: 5.0331
Epoch [35/50] | Loss: 5.0271
Epoch [36/50] | Loss: 5.0212
Epoch [37/50] | Loss: 5.0145
Epoch [38/50] | Loss: 5.0068
Epoch [39/50] | Loss: 5.0005
Epoch [40/50] | Loss: 4.9926
Epoch [41/50] | Loss: 4.9871
Epoch [42/50] | Loss: 4.9807
Epoch [43/50] | Loss: 4.9741
Epoch [44/50] | Loss: 4.9673
Epoch [45/50] | Loss: 4.9591
Epoch [46/50] | Loss: 4.9523
Epoch [47/50] | Loss: 4.9445
Epoch [48/50] | Loss: 4.9370
Epoch [49/50] | Loss: 4.9297
Epoch [50/50] | Loss: 4.9215
```

In [222…

```python
# After training, we can now do zero-shot prediction
@torch.no_grad()
def predict_best_match(model, query_input, candidate_inputs, device):
    model.eval()

    query_feat = model.image_encoder(query_input.unsqueeze(0).float().to(device))
    query_emb = F.normalize(model.img_projector(query_feat), dim=1)

    cand_feat = model.audio_encoder(candidate_inputs.float().to(device))
    cand_emb = F.normalize(model.aud_projector(cand_feat), dim=1)
```

```
    scores = torch.einsum('id, jd -> ij', query_emb, cand_emb)

    best_match_idx = scores.argmax().item()

    print(f"Best match: {best_match_idx} with score {scores[0, best_match_idx].item()}")

    return best_match_idx, scores
```

In [223…
```
m_a, m_b, l_x = next(iter(testdata))
```

In [224…
```
for i in range(5):
    fig, axes = plt.subplots(1, 3, figsize=(5, 3))

    # Label as text
    axes[0].text(0.5, 0.5, str(l_x[i].item()), fontsize=14,
                 ha='center', va='center')
    axes[0].axis('off')

    # Image
    img = m_a[i]
    if img.dim() == 2:  # [H, W]
        img_array = img.numpy()
    else:  # [1, H, W] -> squeeze
        img_array = img.squeeze().numpy()
    axes[1].imshow(img_array, cmap='gray')
    axes[1].axis('off')
    axes[1].set_title("Image")

    # Audio image
    audio_img = m_b[i]
    if audio_img.dim() == 2:
        audio_array = audio_img.numpy()
    else:
        audio_array = audio_img.squeeze().numpy()
    axes[2].imshow(audio_array, cmap='magma')
    axes[2].axis('off')
    axes[2].set_title("Audio Image")

    plt.tight_layout()
    plt.show()
```

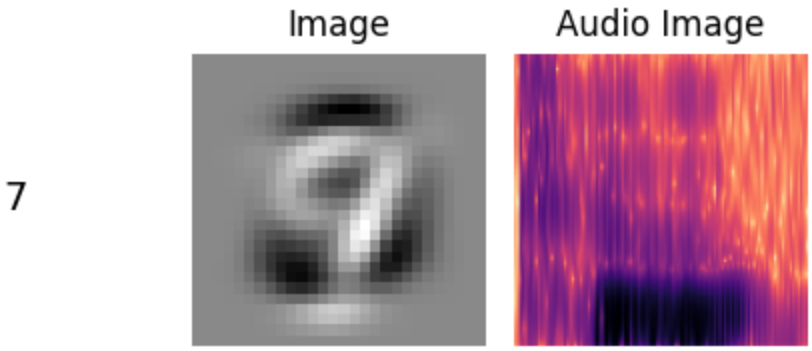


Image Audio Image

2

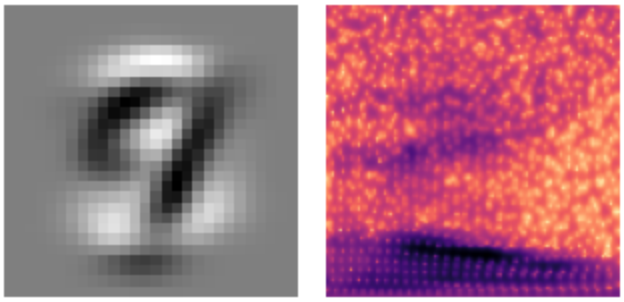


Image Audio Image

1

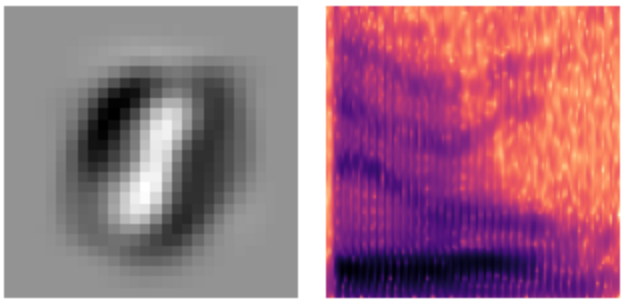
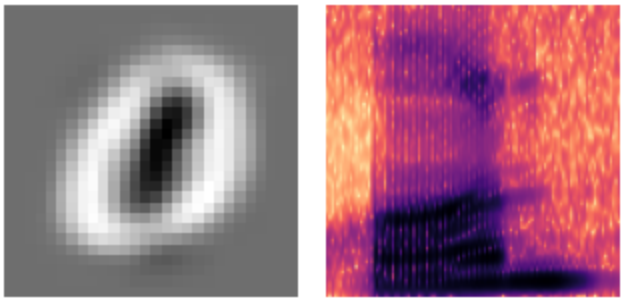
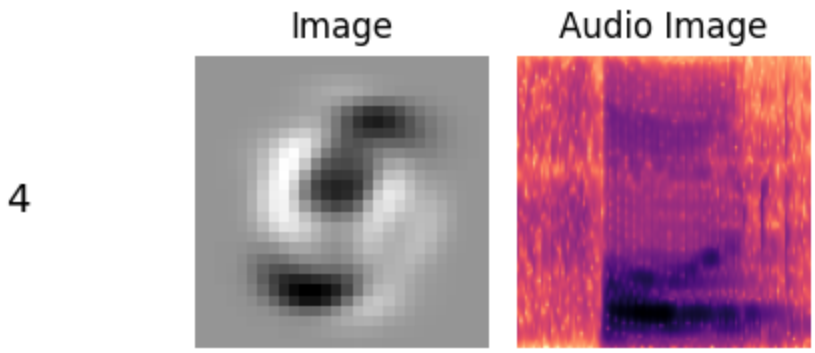


Image Audio Image

0





In [227…
```
predict_best_match(model, m_a[201], m_b, device='cuda')
```

```
Best match: 22 with score 0.2947305142879486
```

Out[227…
```
(22,
 tensor([[ 0.1198,  0.0942, -0.0117,  0.0712,  0.1218, -0.0043,  0.0749, -0.0215,
           0.1093,  0.0382, -0.0685,  0.0712,  0.0504,  0.2093,  0.0021, -0.0423,
           0.1269, -0.0642, -0.0733,  0.0894,  0.1321,  0.0176,  0.2947, -0.0115,
           0.1460, -0.0421,  0.0532, -0.1761,  0.1267,  0.0367, -0.0719,  0.0530,
           0.0703,  0.0614,  0.0789,  0.0545,  0.0050,  0.1103,  0.0818,  0.1283,
          -0.0125,  0.0672,  0.0172, -0.0294,  0.0978, -0.0631, -0.0290, -0.0304,
          -0.1661,  0.0203,  0.1787,  0.0488, -0.0522,  0.0877,  0.1465,  0.2934,
          -0.1525,  0.0768, -0.1565,  0.0752,  0.0155, -0.0407,  0.0634, -0.0390,
           0.2191,  0.2599, -0.1227, -0.1913,  0.1850,  0.0011, -0.0105,  0.0985,
           0.1516, -0.0919,  0.1269,  0.1197,  0.0273,  0.1360,  0.1816,  0.1296,
          -0.0174,  0.1579, -0.0180, -0.0617, -0.0444, -0.0400,  0.0915, -0.0888,
          -0.0654,  0.0311,  0.2379,  0.1479, -0.0423,  0.1276,  0.0176, -0.0401,
           0.2243, -0.0131,  0.0547,  0.0817,  0.0293, -0.0778, -0.0262, -0.0850,
           0.0285,  0.1648,  0.0093,  0.0464,  0.0057,  0.0189,  0.1482,  0.1653,
          -0.0355, -0.0476, -0.0816,  0.0166,  0.0595, -0.1726,  0.1709,  0.0428,
           0.0210,  0.0056,  0.1926,  0.0055, -0.0370,  0.1758,  0.0439,  0.0184,
           0.2252,  0.1859,  0.0866,  0.0501,  0.1539, -0.1481, -0.0142,  0.0016,
           0.1237,  0.0805,  0.1758, -0.0663,  0.1517,  0.1683,  0.1364, -0.1046,
           0.0722, -0.0116, -0.0038,  0.0957,  0.0366, -0.1797,  0.0854,  0.0949,
          -0.0081, -0.0011, -0.0294,  0.0605,  0.1328,  0.1040, -0.0539, -0.0508,
           0.2228,  0.0324, -0.0248, -0.0073,  0.2175, -0.0625,  0.1014,  0.0520,
           0.0042, -0.0312,  0.1276, -0.0517,  0.1745, -0.0509,  0.0754, -0.0272,
           0.0437,  0.0075,  0.0171,  0.1046,  0.2057, -0.0237,  0.0021,  0.0698,
          -0.0443,  0.0163, -0.0451,  0.0309,  0.0320,  0.0830, -0.0592,  0.0324,
          -0.0178,  0.0652,  0.0819, -0.0313,  0.0497,  0.1357, -0.1395, -0.0225,
          -0.0333,  0.1246,  0.0401,  0.1068,  0.1095,  0.0354, -0.0096, -0.0508,
           0.0689,  0.0404,  0.0138,  0.0619,  0.0608,  0.0442,  0.1471, -0.0791,
           0.1111,  0.2092, -0.0064,  0.0606, -0.0206,  0.1742, -0.0515, -0.0729,
           0.1444,  0.1266,  0.0634,  0.0912,  0.0630,  0.0026,  0.0559, -0.1252,
          -0.0637, -0.1294, -0.1114,  0.0216,  0.1704, -0.0091, -0.0587,  0.0597,
          -0.0325,  0.0350, -0.0285,  0.0745, -0.0016, -0.0026,  0.0349,  0.0365,
           0.0374,  0.0404, -0.0916, -0.1302,  0.0402,  0.0503,  0.0075,  0.0197]],
         device='cuda:0'))
```

In [228…
```
l_x[22], l_x[201]
```

Out[228…
```
(tensor(6), tensor(6))
```

IT WORKS! Although not completely consistent (there is more room for improvement of the models), we are able to see that the audio that maps to 5 best lines up with the image of 5.

Now answer some of these questions:

1. (5 points) Any suprising results from using this on your dataset?

I ended up not using my own dataset, so I do not have particularly suprising results.

2. (5 points) Typically, cross-entropy loss is used in this contrastive learning, why is this the case?

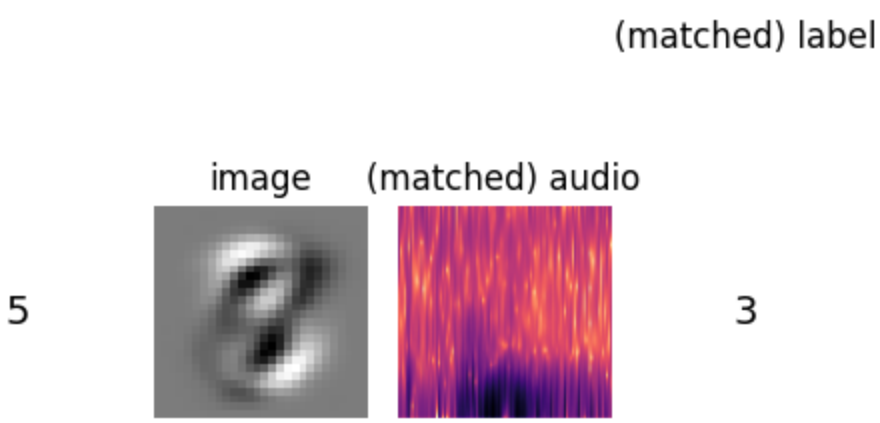
Cross-entropy loss is used as it is equivalent to the InfoNCE loss. Since each batch has a single correct match, we can essentially treat this as a multiclass problem, where we aim to classify correctly the matching image to audio, for each image and each audio. Thus, the cross entropy punishes all negative pairs and encourages positive pairs. Also, we use the loss function from one modality to the other in both directions, as to not create imbalance.

3. (10 points) Create some visual examples of the data post alignment. Can you point out samples where the alignment worked and where it failed? Why do you suspect that is?

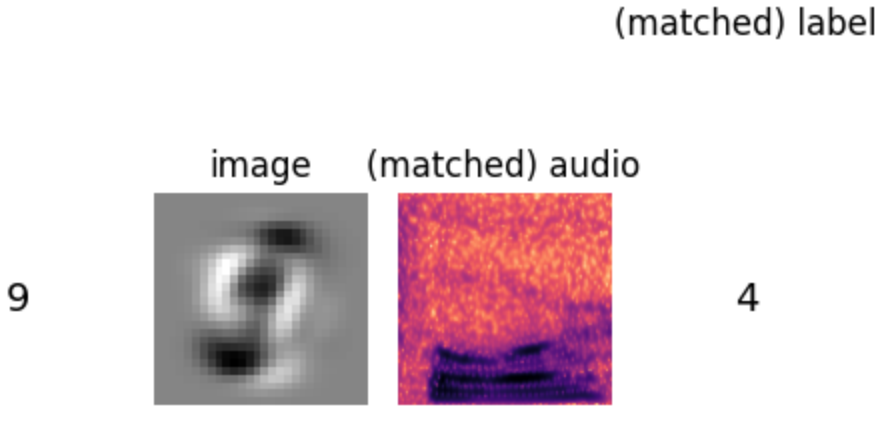
In [229…

```
for i in range(15,20):
  idx, _ = predict_best_match(model, m_a[i], m_b, device='cuda')
  fig, axes = plt.subplots(1, 4, figsize=(5, 3))

  # Label as text
  axes[0].text(0.5, 0.5, str(l_x[i].item()), fontsize=14, ha='center', va='center')
  axes[0].axis('off')

  # Image
  img = m_a[i]
  if img.dim() == 2:  # [H, W]
      img_array = img.numpy()
  else:  # [1, H, W] -> squeeze
      img_array = img.squeeze().numpy()
  axes[1].imshow(img_array, cmap='gray')
  axes[1].axis('off')
  axes[1].set_title("image")

  # matching audio image
  audio_img = m_b[idx]
  if audio_img.dim() == 2:
      audio_array = audio_img.numpy()
  else:
      audio_array = audio_img.squeeze().numpy()
  axes[2].imshow(audio_array, cmap='magma')
  axes[2].axis('off')
  axes[2].set_title("(matched) audio")


  # Label as text
  axes[3].text(0.5, 0.5, str(l_x[idx].item()), fontsize=14, ha='center', va='center')
  axes[3].axis('off')
  axes[3].set_title("(matched) label")

  plt.tight_layout()
  plt.show()
```

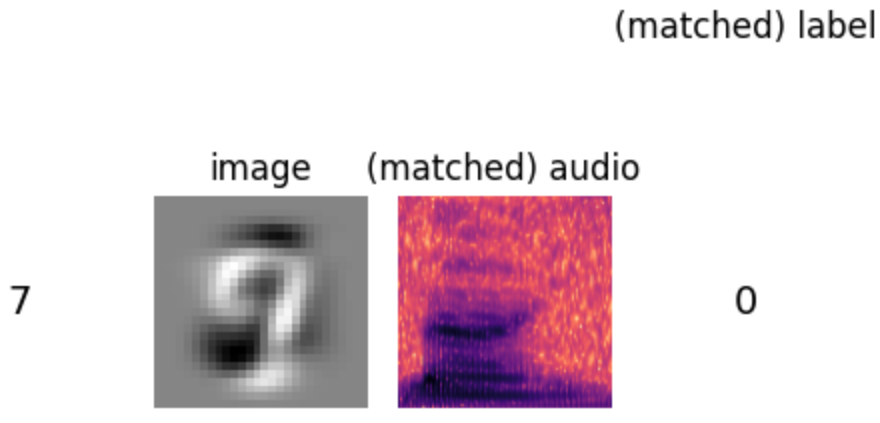
```
Best match: 213 with score 0.28304120898246765
```



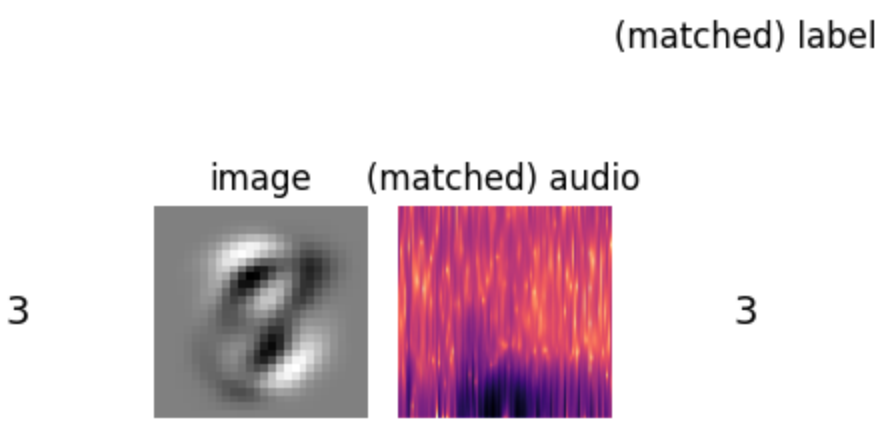

```
Best match: 210 with score 0.3039979636669159
```



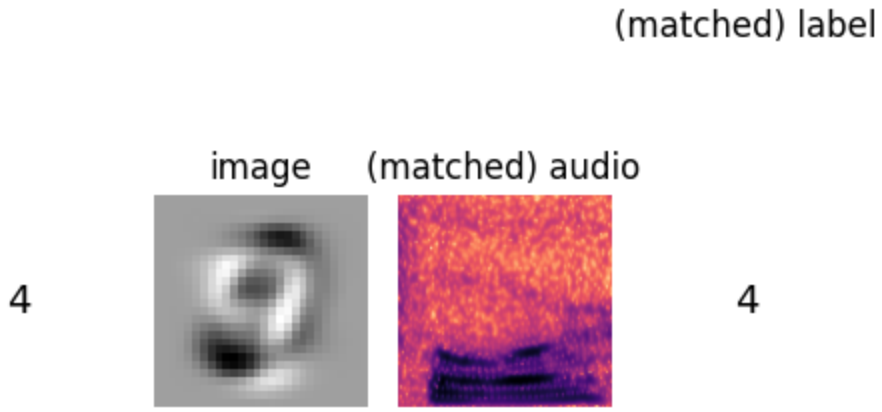

```
Best match: 192 with score 0.27412277460098267
```




```
Best match: 213 with score 0.30859145522117615
```




```
Best match: 210 with score 0.28616493940353394
```



We see that in a couple of cases, we recovered audios with labels that matched the image labels. This shows that our model is capable of aligning the embeddings to similar modalities. In the cases where our model fails, some of them could be close to eachother, such as 9 and 4, for example. I think there are easier classes to align than others, depending on how distinct the image/audio is. Thus, we see the model can differentiate between the latent space representations of some numbers better than others.

Problem 7: Reflection (10 points)

Now we'll take some time to reflect on this homework. Take some time to discuss the following:

1. (5 points) What concept did you find the most interesting?

I think the ideas of late vs early fusion is very interesting. Something that I had never been exposed to is tensor fusion, which seems like it could be a very useful method. In the past, I have only used concatenation as a fusion method, so exploring some new ideas was good to integrate into my own personal research.

2. (5 points) Which concepts (if any) do you see being useful towards your goal? Why? If there was none, discuss why.

As mentioned above, I am looking forward to trying new forms of early fusion in my own research, as I currently only use concatenation. Also, I am planning on trying contrastive learning in my own research, so coding it from scratch helped a lot.

3. (0 points, optional) Is there a topic that was discussed during lectures up to the release of the assignment that you wished was covered in the homework? Any from the assignment that you wanted there to be touched upon more? Any feedback you have in general for homeworks or the class?

I think the homeworks in general can be very vague in instruction and dense in terms of work outside the scope of the lectures, etc. For example, it would have been good to have some more notes on the low-rank tensor fusion, or a more standard setup for the ways to measure memory usage and time to convergence (for unimodal models, we can't easily use the code setup in the MultiBench training script). Also, just generally a Piazza where we can better ask these types of questions.

In [ ]: